Computational Holography and Phase Retrieval: High-Performance Mathematical Pipelines in a C# Ecosystem

The generation and processing of computer-generated holography (CGH) represent a multidisciplinary challenge at the intersection of wave optics, non-convex optimization, and high-performance computing. Reconstructing a three-dimensional light field requires the accurate numerical simulation of scalar wave propagation and the extraction of phase-only representations suitable for Spatial Light Modulators (SLMs). When scaled to dynamic, multi-layered, or time-sequenced representations—such as those encoded in multi-frame Graphics Interchange Format (GIF) or high-precision Tagged Image File Format (TIFF) files—the computational demands necessitate a meticulously optimized software harness.
The successful implementation of such a system requires bridging complex theoretical physics with modern software architecture. By leveraging a C# ecosystem augmented with GPU acceleration through ILGPU and CUDA, robust image decoding algorithms via LibTiff.Net and SixLabors.ImageSharp, and asynchronous memory management using bounded channels, it is possible to execute complex algorithmic phase retrievals in real-time. This report details the comprehensive architectural, mathematical, and algorithmic paradigms required to build an industrial-grade CGH pipeline.

Mathematical Foundations of Scalar Wave Propagation

The cornerstone of computational holography is the accurate numerical simulation of free-space light propagation. To project a target intensity distribution at a specific spatial depth, the optical system must reverse-engineer the required phase pattern at the SLM plane. This physical process is governed by the scalar Helmholtz equation, which dictates how electromagnetic fields propagate through a homogeneous medium.

The Rayleigh-Sommerfeld Diffraction Integral

For a fully coherent monochromatic scalar wave field, free-space propagation can be described without relying on paraxial approximations using the Rayleigh-Sommerfeld (RS) diffraction integral[1]. Given a complex optical field at a source plane, the diffracted field at a parallel observation plane is calculated by evaluating the contributions of secondary spherical wavelets emitted from every point on the source aperture.
The field is expressed as a continuous convolution integral, where the output wavefield is a superposition of all incoming point-spread functions. The exact mathematical formulation defines the propagated field $U(x, y, z)$ as a function of the source field $U(x_1, y_1, 0)$ over

the aperture geometry[2]. The integral incorporates the wavenumber $k = 2\pi/\lambda$, where λ is the wavelength, and computes the Euclidean distance r between the source coordinate and the observation coordinate. An obliquity factor, typically $\cos(\theta) = z/r$, ensures the appropriate geometric scaling of the propagating wave[3].

While the RS integral provides exact analytical solutions in the spatial domain, its direct numerical integration—often referred to as the RS direct integration (RS-DI) method—exhibits an insurmountable computational complexity. For a discretized grid of size $N \times N$, resolving the direct integration demands $\mathcal{O}(N^4)$ floating-point operations[4]. This exponential scaling renders the RS-DI method entirely unsuitable for real-time iterative phase retrieval algorithms, necessitating a transformation into the spatial frequency domain.

The Angular Spectrum Method

To achieve computational tractability, the Rayleigh-Sommerfeld equation is transformed using the convolution theorem, yielding the Angular Spectrum Method (ASM). The ASM decomposes the source optical field into a linear combination of plane waves, propagates each individual plane wave by multiplying it with a frequency-dependent transfer function, and subsequently reconstructs the field in the spatial domain via an inverse Fourier transform[2].

The initial angular spectrum of the source field is obtained via the two-dimensional Fourier transform, integrating over the source spatial coordinates (x_1, y_1) to map into the spatial frequency coordinates (f_x, f_y) . Propagation over a longitudinal distance z is achieved by multiplying the angular spectrum by the free-space coherent transfer function (CTF). This transfer function dictates the phase accumulation of each plane wave component as it traverses free space:

$$H(f_x, f_y, z) = \begin{cases} \exp\left(j2\pi z\sqrt{\lambda^{-2} - f_x^2 - f_y^2}\right), & \text{for } f_x^2 + f_y^2 \leq \lambda \\ 0, & \text{otherwise} \end{cases}$$

The propagated field is then recovered through an inverse Fourier transform of the modulated spectrum. By mapping the propagation problem into the frequency domain, the computational complexity of the ASM becomes strictly dictated by the efficiency of the Fast Fourier Transform (FFT) algorithm, reducing the operational burden from $\mathcal{O}(N^4)$ to $\mathcal{O}(N^2 \log_2 N)$ [6]. This profound reduction in complexity is what enables modern iterative holography.

Asymmetric Point Spread Functions and Tilted Planes

Traditional ASM assumes that the source plane and the observation plane are perfectly parallel.

However, in advanced near-eye displays and augmented reality headsets, the holographic projection often must map onto tilted or arbitrary focal planes[8]. Simulating propagation between non-parallel planes introduces significant geometric complexity.
To address this, the optical pipeline utilizes an asymmetric Point Spread Function (PSF). The coordinate system of the tilted plane (x', y', z') is mapped to the global coordinate system (x, y, z) using a rotation matrix derived from the Rodrigues rotation formula[8]. This orthogonal transformation prevents gimbal lock and allows the precise calculation of the optical path length r' across the rotated space.

Because the rotated plane implies a non-zero, continuously varying depth z, an initial carrier wave $u_0 = \exp(jkz)$ is introduced. The wavefield on the tilted plane generated by a source point is then characterized by an asymmetric PSF[8]:

$$U'_{asym}(x', y') = u_0 \cdot \frac{1}{j\lambda r'} \exp[jk(r' + r_7 x' + r_8$$

where r_7 and r_8 are components of the rotation matrix defining the tilt[8]. Transforming the spectrum from the local tilted system back to the global system requires calculating a Jacobian determinant to handle the unequal spacing of the new frequency coordinates, followed by interpolation to execute the standard FFT-based ASM[8]. This mathematical extension permits true 3D holographic rendering across complex, multi-planar environments.

Overcoming Sampling Limits in Frequency Space

Despite its algorithmic efficiency, the standard ASM is severely constrained by numerical sampling limits. The transfer function $H(f_x, f_y, z)$ contains a highly oscillatory chirp phase term. As the propagation distance z or the spatial frequency increases, the oscillation frequency of the exponential term accelerates rapidly[7].
If the local frequency of this phase exceeds the Nyquist frequency of the discrete computational grid, numerical aliasing occurs. Aliasing causes high-frequency components to wrap around the discrete boundary, artificially contaminating the low-frequency spectrum and generating destructive interference artifacts that ruin the holographic reconstruction[7].

Band-Limited Angular Spectrum Method

To prevent this aliasing phenomenon, the Band-Limited Angular Spectrum Method (BLASM) imposes a strict spatial-frequency cutoff. The highest spatial frequency required for a reliable propagation simulation is constrained by the geometry of the source and observation windows[7].

The required band of the spectrum can be determined by analyzing the maximum angle of diffraction required to fill the observation window from the source aperture:

$$u_{need} = \frac{1}{\lambda}\left[\left(\frac{2z}{W_x + S_x}\right)^2 + 1\right]^{-1/2}$$

where W_x is the width of the input source aperture and S_x is the width of the destination observation area[7]. To maintain strict adherence to the Nyquist-Shannon sampling theorem, the maximum correctly sampled bandwidth is bounded by $u_{max} \leq \frac{1}{\lambda}[(2z\Delta u)^2 + 1]^{1/2}$. By applying a low-pass rectangular window to the transfer function, BLASM mathematically suppresses the under-sampled, highly oscillatory components[7]. This explicit bandwidth truncation eliminates aliasing and successfully extends the validity of the ASM from the extreme near-field into the deeper Fresnel and Fraunhofer diffraction regimes[7].

Scaled Angular Spectrum Method

A fundamental limitation of the conventional ASM and BLASM is that the pixel pitch (sampling interval) of the observation plane must remain identical to that of the source plane. This one-to-one mapping severely restricts the ability to simulate large far-field patterns from small SLM apertures[4].

The Scaled Angular Spectrum Method (Sc-ASM) introduces a scaling factor $\alpha = \Delta x / \Delta u$ to decouple the source and observation resolutions[4]. The discrete Fourier components are modulated by this scaling factor, allowing the output wavefield to be calculated over a varying window size without requiring massive zero-padding arrays. The scaled algorithm modifies the inverse Fourier transform into a chirp z-transform or scaled discrete Fourier transform (Sc-DFT), enabling the pipeline to independently adjust the observation pixel pitch while maintaining the $\mathcal{O}(N^2 \log_2 N)$ computational profile[4]. This scaling is highly critical when mapping a dense $4\mu m$ pixel SLM to a wide field-of-view projection plane.

Iterative Phase Retrieval Algorithms

The physical generation of a hologram requires translating a desired target intensity pattern into a pure phase map. Because modern SLMs, as well as conventional detectors and the human retina, operate predominantly in the intensity domain, the underlying phase distribution must be synthetically engineered[12]. Finding a pure phase distribution at the SLM plane that generates a highly specific amplitude at the target plane constitutes a strictly non-linear, ill-posed inverse problem[12].

The Gerchberg-Saxton Algorithm

The Gerchberg-Saxton (GS) algorithm serves as the historical and foundational Fourier iterative algorithm for optimizing phase-only computer-generated holograms[14]. The GS framework relies on the method of alternating projections. The algorithm rapidly bounces the wavefield back and forth between the SLM plane and the target image plane, enforcing specific amplitude constraints at each boundary while allowing the phase component to evolve organically[14].
The iterative process follows a cyclic pattern. First, a random uniform phase distribution is assigned to the target amplitude, establishing the initial complex field. Second, this field is backward-propagated to the SLM plane using the inverse ASM. Third, the resulting amplitude at the SLM plane is discarded and replaced with a uniform amplitude (matching the physical illumination of the reference laser), while the computed phase is preserved. Fourth, this phase-only field is forward-propagated to the target plane using the ASM. Finally, the resulting target amplitude is replaced with the ideal desired amplitude, and the cycle repeats[16].
While computationally lightweight per iteration, the traditional GS algorithm suffers from severe stagnation issues. The algorithm is highly sensitive to the initial random phase guess and is notoriously prone to trapping itself in local optical optima[18]. This stagnation manifests physically as severe speckle noise and distinct vortex artifacts in the reconstructed image.

Layer-Oriented 3D Holography and Cross-Talk

For three-dimensional scenes, the GS algorithm is expanded using an angular-spectrum layer-oriented approach[17]. The volumetric 3D scene is sliced into multiple distinct depth layers. During forward propagation, the algorithm computes the complex amplitude of each individual depth layer, superimposes them linearly, and extracts a unified phase[17].
However, as the number of layers increases, the direct superimposition introduces massive inter-layer crosstalk. Because light diffracted for a far-field layer must pass through the focal volume of a near-field layer, the presence of these defocal effects introduces destructive noise where the focal and defocal regions overlap[17]. To mitigate this, hybrid iterative ASM frameworks integrate the layer superposition into a solitary optimization loop, distributing the error energy across the layers to minimize overall speckle[15].

Compressive Sensing Gerchberg-Saxton

To drastically optimize convergence times for sparse targets or point-cloud data, the Compressive Sensing Gerchberg-Saxton (CS-GS) variant is employed[16]. Traditional GS requires full forward and backward FFTs over the entire SLM grid M for every iteration K, scaling as $\mathcal{O}(K \times N \times M)$ where N is the number of target spots.
Operating under the mathematical assumptions of compressive sensing and k-space compressed imaging, CS-GS evaluates only a small, randomly distributed subset of cM pixels during the early stages of optimization, where $c \ll 1$ [16]. Because the system is highly under-determined, exact solutions are unnecessary in early iterations. The computational cost is effectively slashed to roughly $N \times M$, bridging the gap between random superposition algorithms and fully converged GS algorithms, providing real-time projection capabilities for

dynamic point-cloud scenes[16].

Phase Retrieval Algorithm	Mathematical Paradigm	Key Advantage	Primary Drawback
Traditional GS	Alternating Projections	Simple to implement, fast single-iteration times	Prone to local optima, high speckle noise
Layer-Oriented GS	Multi-plane ASM Superposition	Allows true 3D volumetric depth cues	Severe inter-layer defocal crosstalk
Compressive Sensing GS	Sparse Matrix Sampling	Extremely fast execution for point clouds	Ineffective for dense, continuous-tone images
Wirtinger Flow (WF)	Non-Convex Gradient Descent	Global optimization, superior image fidelity	High computational overhead, requires step tuning

Non-Convex Optimization via Wirtinger Flow

To overcome the inherent local-minima stagnation characteristic of alternating projection methods, modern computational holography increasingly frames phase retrieval as a non-convex empirical optimization problem solved via gradient descent. The Wirtinger Flow (WF) algorithm provides a mathematically rigorous alternative to GS by optimizing a defined fidelity metric directly[13].

Because the holographic phase argument is represented as a complex vector $z \in \mathbb{C}^n$, the error function $f(z)$ is non-holomorphic, meaning it fails the Cauchy-Riemann equations and is not differentiable in the traditional complex sense[13]. The WF algorithm bypasses this mathematical barrier using complex Wirtinger derivatives. For a cost function $f(z, \bar{z})$ of a complex variable and its conjugate, the gradient is formally defined by treating z and $\bar{z}$ as independent variables:

$$\nabla f(z) = (\nabla_z f)^T dz + (\nabla_{\bar{z}} f)^T d\bar{z}$$

In the context of phase retrieval derived from intensity measurements $y_i = |\langle a_i, z \rangle|^2$, the empirical risk minimization is formulated as an error metric $f(z) = \frac{1}{2m} \sum_{i=1}^{m} (y_i - |a_i^H z|^2)^2$ [21]. The precise Wirtinger gradient step used to update the phase representation takes the following form:

$$\nabla f(z) = \frac{1}{m} \sum_{i=1}^{m} \left(y_i - |a_i^H z|^2 \right) (a_i a_i^H) z$$

The algorithm iteratively steps down the gradient landscape updating the estimate $z_{k+1} = z_k - \mu_k \nabla f(z_k)$, where μ_k represents an evolving step size that prevents divergent oscillation[21].

Median Reweighted Wirtinger Flow for Turbulence Mitigation

In environments characterized by high noise, occlusion, or atmospheric turbulence—such as long-distance holographic communication or biological tissue propagation—standard WF can be derailed by extreme gradient outliers. The Median Reweighted Wirtinger Flow (MRWF) is utilized to enforce robust convergence[21].

MRWF introduces a statistical weighting factor during the initialization phase, explicitly defining $\lambda_0 = \text{med}(y)/0.455$, where $\text{med}(y)$ is the median of the observed intensity vector[21]. By applying a weighting factor to the gradient steps, MRWF heavily penalizes extreme outliers caused by atmospheric phase distortion, preventing inter-mode crosstalk in Orbital Angular Momentum (OAM) multiplexing and Laguerre-Gaussian (LG) beam projection[21]. The MRWF typically reaches optimal phase recovery within roughly 120 iterations, demonstrating profound resilience against turbulent phase screens compared to traditional GS variants[21].

Convergence Evaluation Metrics

The fidelity of the phase retrieval process is continuously evaluated using standardized numerical metrics to quantify reconstruction quality[3].

Evaluation Metric	Description	Target Value for High Fidelity

NMSE (Normalized Mean Square Error)	Quantifies the absolute pixel-wise variance between target and reconstructed intensity[3].	Approaching 0.0
PSNR (Peak Signal-to-Noise Ratio)	Logarithmic measurement of image degradation, comparing the maximum possible signal to the compressing noise[3].	$>$ (Higher is better)
SSIM (Structural Similarity Index)	Perceptual metric evaluating luminance, contrast, and structure, mimicking human visual perception[3].	Approaching 1.0

While GS algorithms often plateau quickly at lower PSNR thresholds due to speckle, Wirtinger Flow continuous strictly minimizing the NMSE, resulting in structurally superior SSIM scores and sharper holographic edge reconstruction[18].

Double-Phase Amplitude Encoding for SLM Hardware

Even with mathematically perfect phase retrieval, physical optical hardware imposes strict limitations. Standard Spatial Light Modulators (SLMs) are typically phase-only devices, utilizing twisted nematic liquid crystals to retard the light field. This means arbitrary complex fields $H_1 = A\exp(j\phi)$ containing both amplitude and phase variation cannot be natively displayed on a single panel[23].

Double-phase amplitude encoding provides a sophisticated mathematical workaround for this hardware limitation. The algorithm maps a single complex value into two separate, strictly phase-only values[25]. By grouping adjacent SLM pixels into closely coupled "macropixels," the local amplitude is encoded as a phase difference between the two adjacent pixels, while the overall macroscopic phase is encoded as their average phase shift.

During free-space propagation, the localized diffraction from these two sub-pixels causes them to interfere in the far-field. This highly controlled destructive and constructive interference mathematically reproduces the original target amplitude and phase with exact precision, effectively granting a phase-only SLM the capabilities of a highly expensive complex-modulation SLM[23].

Architecting the C# GPU Computation Harness

Executing these highly iterative, Fourier-intensive algorithms—requiring billions of floating-point operations per second for 4K video holography—is entirely prohibitive on standard CPU architectures. The C# ecosystem provides an elegant, object-oriented framework for managing

the overarching pipeline architecture, while strictly delegating the mathematical heavy lifting to the Graphics Processing Unit (GPU).

GPU Kernel Acceleration via ILGPU

ILGPU is a Just-In-Time (JIT) compiler written entirely in C# that translates .NET Microsoft Intermediate Language (MSIL) directly into NVIDIA PTX (Parallel Thread Execution) assembly or OpenCL C[28]. This paradigm allows optical engineers to write high-performance kernel code natively in C# without relying on cumbersome native C++ wrappers, fully preserving the type safety, garbage collection tooling, and debugging capabilities of the .NET ecosystem[28].

In the context of the Wirtinger Flow and GS algorithms, ILGPU excels at applying massive element-wise operations: complex multiplications, rapid exponential phase transformations, and pixel-level hardware constraints. A typical ILGPU kernel utilizes an Index2D struct mapped directly to the hardware thread blocks. The memory is accessed via an ArrayView2D<Complex, Stride2D.DenseY> structure, which guarantees contiguous memory access patterns essential for GPU memory bandwidth optimization[28].

Because ILGPU natively supports custom value types like struct-based complex numbers, it entirely circumvents the overhead of object boxing and unboxing on the managed .NET heap[28].

Kernels for applying amplitude targets or executing the Sc-ASM scaling factor α are JIT-compiled at runtime and executed in parallel across millions of unified shading cores[29].

cuFFT Integration for Spectral Processing

While ILGPU expertly handles element-wise mapping, the core of the Angular Spectrum Method relies on extremely efficient two-dimensional Fast Fourier Transforms. The NVIDIA cuFFT library serves as the indispensable standard for these operations, utilizing highly optimized Cooley-Tukey principles that leverage the specific memory architectures of modern streaming multiprocessors[6].

Interfacing the managed C# environment with the unmanaged cuFFT binary requires specialized bindings, often facilitated through ManagedCUDA or native DllImport interoperability[30]. The architectural strategy is to keep the massive wavefront data strictly confined within the GPU's VRAM. Pointers derived from ILGPU MemoryBuffer objects are passed directly to cuFFT execution functions, ensuring zero latency from PCIe bus transfers between the element-wise constraint steps and the transform steps[34].

For 3D layer-oriented holography, the system invokes cufftPlanMany[6]. This API permits batched execution, allowing the GPU to transform dozens of 2D depth layers simultaneously rather than sequentially spinning up new FFT plans[6]. When executing a forward ASM propagation, the pipeline executes a complex-to-complex transform via cufftExecC2C (for single precision) or cufftExecZ2Z (for double precision)[6]. Furthermore, when transforming real-only target amplitudes, cufftExecR2C allows for massive memory savings by only computing half the Hermitian symmetric spectrum, though it requires precise memory padding layouts to ensure compatibility for in-place transforms[6].

Multi-Node Slab Decomposition

For massive computational datasets that exceed the memory limits of a single GPU—such as

massive LiDAR point clouds or ultra-high-resolution photolithography masks—the C# harness orchestrates multi-node execution via slab decomposition[33]. The real-space optical field is divided into p distinct slabs along the slowest-changing spatial axis.

Each GPU node receives a partial slice of the wavefield via cudaMemcpy. The system performs batched 2D real-to-complex FFTs along the local axes. Then, utilizing NVLink and CUDA-aware MPI protocols, the data undergoes a massive local transpose operation, shuffling data across the GPUs to align the remaining axis for a final 1D complex-to-complex cuFFT operation[33]. This complex choreography allows the C# harness to scale phase retrieval infinitely across compute clusters[33].

High-Precision Image Ingestion and Memory Pipelines

The source material feeding the CGH engine heavily dictates the input-output (I/O) architecture. Complex optical targets, such as topographical depth maps and layered phase fields, are stored in high-precision, uncompressed formats like 32-bit floating-point TIFFs. Conversely, dynamic holographic projections utilize sequence-encoded formats like animated GIFs. Safely parsing, extracting, and loading these files into GPU memory requires specialized C# pipelines to avoid memory fragmentation.

Processing 32-bit Floating-Point TIFF Data

Standard 8-bit image formats (like PNG or JPEG) lack the dynamic range required for precise depth mapping or optical wavefront preservation, capping values at 256 discrete levels. The Tagged Image File Format (TIFF) circumvents this by supporting 16-bit and 32-bit IEEE floating-point pixel representations (PREDICTOR_FLOATINGPOINT), mapping data directly to the continuous physical parameters of the scalar light field[36].

Within the C# ecosystem, LibTiff.Net provides explicit, low-level access to the TIFF Image File Directory (IFD) tags. Unlike standard .NET bitmaps which attempt to load entire images into RAM—frequently triggering fatal System.OutOfMemoryException crashes for gigapixel topographies—LibTiff.Net processes large files iteratively in modular chunks[38]. The decoding strategy diverges based on the internal organization of the specific TIFF file:

1. **Stripped TIFFs:** Data is arranged in contiguous horizontal rows. The system utilizes ReadScanline or ReadEncodedStrip to stream data row-by-row into a predefined floating-point array buffer[39].
2. **Tiled TIFFs:** Modern high-performance imaging utilizes tiled formats (e.g., 256×256 pixel chunks) to heavily optimize spatial locality and cache hit rates. Iterating ReadScanline over a tiled TIFF results in extreme I/O overhead due to constant disk seeking. Instead, the harness identifies the structure via Tiff.IsTiled and employs ReadEncodedTile to extract the fully decompressed byte arrays for a specific regional block[38].

Following extraction, the raw byte array is rapidly mapped into a floating-point matrix using Buffer.BlockCopy[42]. LibTiff.Net automatically handles underlying machine endianness and natively processes BigTIFF specifications, granting the system the ability to seamlessly read

datasets exceeding the traditional 4GB file limit[38]. Extracting a continuous float[] span from these structures allows the C# harness to execute a direct, pinned-memory transfer to the GPU, bypassing the garbage-collected Managed Heap and preventing CPU throttling.

Processing Multi-Frame Holograms (Animated GIF)

For dynamic holographic projection—such as rendering rotating objects, fluid dynamics, or time-multiplexed speckle-reduction phase plates—the system must rapidly ingest animated GIFs or APNGs. The SixLabors.ImageSharp library serves as the optimal integration for this workload. Engineered as a fully managed, SIMD-accelerated imaging library, it is explicitly built for high-throughput .NET 8+ environments[44].

A naive implementation utilizing legacy System.Drawing APIs would force all GIF frames into memory simultaneously, rapidly exhausting the .NET Large Object Heap (LOH) and triggering massive, application-stalling Garbage Collection (GC) sweeps[45]. ImageSharp meticulously circumvents this through its architectural design. It utilizes explicit MemoryAllocator configurations—primarily backed by the standard ArrayPool<T>—to continuously recycle pixel buffers across sequential frames without triggering new allocations[44].

Furthermore, when parsing the ImageFrameCollection of a GIF, ImageSharp inherently manages frame quantization and delta comparisons. It optimizes memory footprint by only evaluating the modified pixels between consecutive frames[46]. Because the output required by the CGH algorithms is typically single-channel grayscale amplitude data, ImageSharp's processing pipeline converts the RGB payload immediately during extraction. The underlying library accelerates this pixel mutation using portable Vector128, Vector256, and Vector512 SIMD intrinsics, processing dozens of pixels concurrently directly on the CPU before they are ever dispatched to the GPU[44].

Proper disposal via using block declarations of ImageSharp instances is enforced rigorously in the C# pipeline, as the pooled contiguous buffers must be returned to the allocator to prevent memory leaks in sustained, real-time holographic video streaming[45].

Ingestion Metric	LibTiff.Net	SixLabors.ImageSharp
Primary File Target	32-bit Floating-Point Geodata / Depth	RGB, Grayscale, Animated GIF/WebP
Data Organization	Deep horizontal Strips, local Tiles	Temporal Frames, continuous buffers
Memory Architecture	Direct byte-level struct reads	ArrayPool<T> with SIMD acceleration
Holographic Use Case	Static, ultra-high resolution topography	Dynamic, time-multiplexed video holography

Asynchronous Concurrency and System Orchestration

Interfacing a rapid, hardware-bound disk I/O process (such as streaming TIFF blocks or decoding GIF frames) with a highly intensive GPU processing kernel (such as iterating Wirtinger Flow gradients) creates a classic software producer-consumer bottleneck.

If the CPU extracts and decodes GIF frames significantly faster than the GPU can execute the required cuFFT transformations, the application will buffer thousands of raw, uncompressed frames in RAM, eventually crashing the application[45]. Conversely, if the GPU is starved while waiting for the disk reader, the highly expensive VRAM and shading units sit completely idle, crippling the real-time frame rate of the holographic display.

Bounded Channels and Backpressure Orchestration

The C# ecosystem resolves this asynchronous bottleneck via the System.Threading.Channels API, which provides highly abstracted, thread-safe message passing[49]. Channels provide a performance profile vastly superior to legacy BlockingCollection models because they integrate seamlessly with modern async/await state machines, entirely avoiding OS-level thread starvation and context-switching overhead[49].

To explicitly prevent memory exhaustion, the C# holographic harness utilizes a carefully tuned Bounded Channel. By specifying BoundedChannelOptions with a strict capacity limit (e.g., buffering a maximum of 10 decoded frames) and setting the behavior to BoundedChannelFullMode.Wait, the system institutes automatic backpressure[45].

1. **The Producer (Image Ingestion):** A dedicated background task decodes the TIFF tiles or GIF frames. As each localized frame is fully decoded into a 1D float[] array, the producer attempts to push the payload via await channel.Writer.WriteAsync(frame). If the channel hits its hard capacity limit, the writer task suspends itself asynchronously[45]. This halts the CPU-side decoding loop, preventing the LOH from flooding with unreleased pixel buffers.
2. **The Consumer (GPU Execution):** Simultaneously, the primary GPU dispatch loop continuously pulls from the queue via await channel.Reader.ReadAsync(). Upon receiving the array, it pins the memory, transfers it to the ILGPU MemoryBuffer, and fires the required Gerchberg-Saxton or Wirtinger Flow iterations via cuFFT[49].

This strict architectural decoupling guarantees that peak systemic throughput is perpetually maintained. The SIMD-accelerated image decoding of ImageSharp operates on the CPU in perfect tandem with the concurrent non-convex optimization kernels executing on the GPU, maximizing overall hardware utilization without risking catastrophic memory fragmentation.

Conclusion

The successful realization of dynamic, high-fidelity computer-generated holography is deeply reliant on bridging the immense gap between complex wave physics and applied software engineering. The optical algorithms driving modern phase retrieval—spanning from the

foundational Rayleigh-Sommerfeld integral to the highly robust, non-convex gradient optimization of the Wirtinger Flow—demand absolute mathematical precision and unprecedented computational scale.

By meticulously nesting these complex mathematical operations inside a modern C# ecosystem, optical engineers achieve a compounding advantage. The architecture harnesses the raw, unadulterated speed of GPU execution through ILGPU and cuFFT integration, while securing the execution safety, asynchronous pipeline orchestration, and advanced memory allocation capabilities inherent to .NET. Through the highly targeted application of System.Threading.Channels to manage asynchronous backpressure, alongside the optimized utilization of LibTiff.Net and ImageSharp for massive data extraction, this software harness demonstrates that computationally overwhelming holography can be executed reliably in real-time. This robust fusion of mathematics and managed code ultimately pushes the boundaries of volumetric holographic displays, advanced optical trapping, and quantitative phase imaging into the realm of practical, industrial deployment.

Works cited

1. Optical aperture area determination for accurate illuminance and, https://d-nb.info/1102083984/34
2. An Angular Spectrum Method for Nonlinear Propagation in ... - arXiv, https://arxiv.org/pdf/2608.07389
3. A two-step tilt compensation method for off-axis holographic displays, https://pmc.ncbi.nlm.nih.gov/articles/PMC13373165/
4. the scaled angular spectrum method for far-field and focal analysis, https://opg.optica.org/optcon/fulltext.cfm?uri=optcon-3-6-935
5. (PDF) Nonlinear ray tracing in focused fields, part 1. Calculation of, https://www.researchgate.net/publication/379325660_Nonlinear_ray_tracing_in_focused_fields_part_1_Calculation_of_3D_focused_wavefields_tutorial
6. 1. Introduction — cuFFT 13.4 documentation, https://docs.nvidia.com/cuda/cufft/index.html
7. Band-limited angular spectrum numerical propagation method with, https://opg.optica.org/abstract.cfm?uri=josaa-29-11-2415
8. Asymmetric point-spread function in the tilted plane, https://cris.vub.be/ws/portalfiles/portal/138704705/121442550.pdf
9. Band-limited angular spectrum method for numerical simulation of, https://pubmed.ncbi.nlm.nih.gov/19997186/
10. HTRSD: Hybrid Taylor Rayleigh-Sommerfeld diffraction, https://vccimaging.org/Publications/Chen2022OE/Chen2022OE.pdf
11. Band-Limited Angular Spectrum Method for Numerical Simulation of, https://opg.optica.org/oe/abstract.cfm?uri=oe-17-22-19662
12. Learned residual Gerchberg-Saxton network for computer generated, https://openreview.net/forum?id=3b76QBOlYW
13. Wirtinger Holography for Near-Eye Displays - UNC Computer Science, https://www.cs.unc.edu/~cpk/data/papers/wirt-holo-sigasia19.pdf

14. Optimization of Phase-Only Computer-Generated Holograms Based, https://www.mdpi.com/2076-3417/10/12/4283
15. A Framework for Iterative Phase Retrieval Technique Integration into, https://www.preprints.org/manuscript/202407.2193
16. Fast Calculation of Computer Generated Holograms for 3D ... - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC6481074/
17. Three-dimensional computer-generated holography based on the, https://opg.optica.org/abstract.cfm?uri=oe-31-23-39169
18. Research on the Wirtinger Flow algorithm based on quadratic, https://www.spiedigitallibrary.org/conference-proceedings-of-spie/12942/1294205/Research-on-the-Wirtinger-Flow-algorithm-based-on-quadratic-distribution/10.1117/12.3005650.pdf
19. Gerchberg-Saxton algorithm and angular-spectrum layer-oriented, https://wulixb.iphy.ac.cn/en/article/doi/10.7498/aps.67.20172464
20. Phase Retrieval via Wirtinger Flow: Theory and Algorithms, https://www.researchgate.net/publication/263699198_Phase_Retrieval_via_Wirtinger_Flow_Theory_and_Algorithms
21. Research of Phase Compensation Methods Based on the Median, https://www.mdpi.com/2304-6732/9/9/619
22. On the relationship between SSIM and PSNR for DCT-based, https://www.techrxiv.org/doi/pdf/10.36227/techrxiv.21725390
23. Non-Iterative Sparse Fourier Holograms Using Perlin Noise - MDPI, https://www.mdpi.com/1424-8220/24/22/7358
24. Practical High-Contrast Holography - arXiv, https://arxiv.org/html/2410.19347v1
25. DCPNet: a dual-channel parallel deep neural network for high, https://opg.optica.org/oe/abstract.cfm?uri=oe-31-22-35908
26. Holographic Near-Eye Displays for Virtual and Augmented Reality, https://www.microsoft.com/en-us/research/wp-content/uploads/2017/05/holo_author.pdf
27. Complex-valued dense atrous neural network for high-quality, https://opg.optica.org/col/abstract.cfm?uri=col-23-12-120501
28. ILGPU JIT Compiler for high-performance .Net GPU programs - GitHub, https://github.com/m4rs-mt/ILGPU
29. Setting Up ILGPU - A Modern GPU Compiler for .Net Programs, https://ilgpu.net/docs/01-primers/01-setting-up-ilgpu/
30. Performing Complex Operations on a GPU in C# using ... - Medium, https://medium.com/@almaskhanwazir/performing-complex-operations-on-a-gpu-in-c-using-managedcuda-and-tensorflow-cbee8126a872
31. The Geek Diver | The Geek Diver, http://thegeekdiver.com/
32. (PDF) Complete Solution of the Fifteen Sliding Tile Puzzles on a, https://www.researchgate.net/publication/362559309_Complete_Solution_of_the_Fifteen_Sliding_Tile_Puzzles_on_a_Personal_Computer
33. Scalable Multi-node Fast Fourier Transform on GPUs - arXiv, https://arxiv.org/pdf/2202.12756
34. FFT Example - MATLAB & Simulink - MathWorks,

https://www.mathworks.com/help/gpucoder/ug/generate-code-using-cuFFT-library.html
35. GPU Computing with CUDA Lecture 8 - CUFFT, PyCUDA, https://www.bu.edu/pasi/files/2011/07/Lecture83.pdf
36. Enum TiffType - BitMiracle.github.io, https://bitmiracle.github.io/libtiff.net/help/api/BitMiracle.LibTiff.Classic.TiffType.html
37. GeoTIFF libtiff.net get elevation data in c# - Stack Overflow, https://stackoverflow.com/questions/43864543/geotiff-libtiff-net-get-elevation-data-in-c-sharp
38. Using The TIFF Library — LibTIFF 4.7.2 documentation - GitLab, https://libtiff.gitlab.io/libtiff/libtiff.html
39. Open huge TIF in .NET and copy parts to new image - Stack Overflow, https://stackoverflow.com/questions/9004149/open-huge-tif-in-net-and-copy-parts-to-new-image
40. libtiff.net/Samples/README.md at master · BitMiracle/libtiff ... - GitHub, https://github.com/BitMiracle/libtiff.net/blob/master/Samples/README.md
41. How to translate Tiff.ReadEncodedTile to elevation terrain matrix, https://stackoverflow.com/questions/37438085/how-to-translate-tiff-readencodedtile-to-elevation-terrain-matrix-from-height-ma
42. How to read GeoTIFF file from C# - Stack Overflow, https://stackoverflow.com/questions/9188572/how-to-read-geotiff-file-from-c-sharp
43. LibTIFF 4.7.2 documentation - GitLab, https://libtiff.gitlab.io/libtiff/functions/libtiff.html
44. ImageSharp - Six Labors, https://sixlabors.com/products/imagesharp/
45. C# Image Manipulation Techniques and Technology - Cloudinary, https://cloudinary.com/guides/image-effects/c-image-manipulation
46. ImageSharp - Six Labors Documentation, https://docs.sixlabors.com/articles/toc.pdf
47. Quantization: reduce memory footprint without compromising speed, https://github.com/SixLabors/ImageSharp/issues/1350
48. Getting Started - Six Labors Documentation, https://docs.sixlabors.com/articles/imagesharp/gettingstarted.html
49. Use System.IO.Pipelines and System.Threading.Channels APIs to, https://dev.to/joni2nja/use-system-io-pipelines-and-system-threading-channels-apis-to-boost-performance-2nj5
50. Channels in C# .NET: Building High-Performance Concurrent, https://www.presidio.com/learn-from-us/blog/channels-in-c-net-building-high-performance-concurrent-pipelines/
51. Channels - .NET - Microsoft Learn, https://learn.microsoft.com/en-us/dotnet/core/extensions/channels
52. How to use System.Threading.Channels in .NET Core - InfoWorld, https://www.infoworld.com/article/2263338/how-to-use-systemthreadingchannels-in-net-core.html